

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1710

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent

architectures, using scalable systems and integrate components in distributed AI applications.

(BTL 5)

Assessment Tool Weightage: 20%

Duration : 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I ___R. Akshaya (192524118)_____ (Name / Reg No) certify
that this submission is my original work and that I

have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in
disciplinary action.

Signature of the student:***R.Akshaya***

Student 3: Akshaya R | Register Number: 192524118

Reverse Engineering a Ride-Hailing Dispatch System

A ride-hailing app matches riders to nearby drivers within seconds, even during peak demand. By observing its

outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how location-data

1. Introduction

A ride-hailing application such as Uber or Ola must match riders with suitable nearby drivers within a few seconds. The system continuously processes large volumes of location updates, ride requests, driver availability, and trip information. By observing the inputs and outputs of such a system, its underlying distributed AI pipeline can be approximated using **Apache Spark RDDs (Resilient Distributed Datasets)**.

The reconstructed system collects driver and rider location data, preprocesses the data, identifies nearby available drivers, calculates a matching score, and selects the most suitable driver for each ride request.

2. Distributed AI Pipeline

The reconstructed ride-hailing dispatch pipeline consists of the following stages:

Location Data → Preprocessing → Partitioning → Nearby Driver Search → Matching Score → Driver Selection → Dispatch

Module 1 – Location Data Collection and Preprocessing

The system receives location information from drivers and riders.

Example driver data:

Driver_ID, Latitude, Longitude, Status

D101, 13.0827, 80.2707, Available

D102, 13.0674, 80.2376, Busy

D103, 13.0878, 80.2785, Available

Example ride request:

Ride_ID, Latitude, Longitude

R001, 13.0800, 80.2750

The raw data is converted into structured records using Spark RDDs. Invalid records can be removed and location values can be normalized.

```
drivers = sc.textFile("drivers.csv")

driver_rdd = (
    drivers
    .map(lambda line: line.split(","))
    .filter(lambda x: len(x) == 4)
    .map(lambda x: (x[0], float(x[1]), float(x[2]), x[3]))
)
```

Only drivers whose status is **Available** are considered for matching.

```
available_drivers = driver_rdd.filter(
    lambda x: x[3].lower() == "available"
)
```

3. Module 2 – Finding Nearby Drivers

The dispatch system needs to determine which available drivers are geographically close to the rider.

A simple distance calculation can be used. For a prototype, Euclidean distance can be calculated using latitude and longitude differences:

```
from math import sqrt
```

```
def distance(driver, rider_lat, rider_lon):
    return sqrt(
        (driver[1] - rider_lat) ** 2 +
        (driver[2] - rider_lon) ** 2
    )
```

For real-world geographic coordinates, the **Haversine formula** would provide a more accurate distance.

The RDD can then calculate the distance of every available driver from the rider:

```
rider_lat = 13.0800
```

```
rider_lon = 13.2750
```

```
nearby = available_drivers.map(  
    lambda d: (d[0], distance(d, rider_lat, rider_lon))  
)
```

A maximum search radius can then be applied:

```
nearby = nearby.filter(lambda x: x[1] < 0.02)
```

This eliminates drivers that are too far away.

4. Module 3 – Driver Matching

The closest driver is not necessarily always the best driver. A practical dispatch system can consider several factors such as:

- Distance from rider
- Driver availability
- Estimated arrival time
- Driver rating
- Vehicle type
- Current workload

For a simplified implementation, distance can be used as the primary matching factor.

```
best_driver = nearby.min(  
    key=lambda x: x[1]  
)
```

The result gives the driver with the smallest distance from the rider.

For example:

Driver D101 → 0.015

Driver D103 → 0.006

Driver D105 → 0.011

The system selects:

Best Driver → D103

5. Use of PySpark RDDs

RDDs are useful because ride-hailing systems must process large amounts of continuously generated data.

Important RDD operations include:

map()

Transforms each record into another format.

```
driver_rdd.map(lambda x: (x[0], x[1], x[2]))
```

filter()

Removes unavailable or invalid drivers.

```
driver_rdd.filter(lambda x: x[3] == "Available")
```

reduceByKey()

Can be used to aggregate information such as the number of available drivers in different regions.

```
region_counts = driver_rdd.map(  
    lambda x: ("Chennai", 1)  
)reduceByKey(lambda a, b: a + b)
```

sortBy()

Can rank drivers according to their distance or matching score.

```
ranked = nearby.sortBy(lambda x: x[1])
```

collect()

Retrieves the processed results.

```
result = ranked.collect()
```

6. Distributed Processing Architecture

The reconstructed architecture can be represented as:

Rider Request

|

v

+-----+

| Data Preprocessing |

+-----+

|

v

Spark Driver

|

+-----+-----+

|

|

v

v

Worker Node 1

Worker Node 2

Driver Locations

Driver Locations

|

|

+-----+-----+

|

v

Distance Calculation

|

v

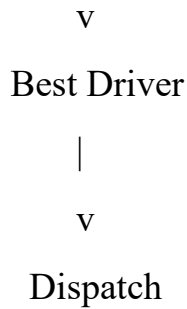
Candidate Drivers

|

v

Ranking / Scoring

|



The driver program coordinates the computation, while Spark worker nodes process partitions of driver-location data in parallel.

7. Reverse Engineering the System

The internal implementation of a commercial ride-hailing application is not directly visible. However, its behavior can be inferred from observable inputs and outputs.

Observable Behavior	Inferred Component
Nearby drivers appear first	Location-based filtering
Unavailable drivers are ignored	Availability filtering
Closest driver is preferred	Distance-based ranking
Large numbers of drivers are processed quickly	Distributed processing
Different drivers receive different requests	Matching/dispatch algorithm
Driver locations continuously change	Real-time location updates

Therefore, the observed behavior suggests a pipeline involving **data ingestion, filtering, spatial matching, ranking, and dispatch**.

8. Simple PySpark Implementation

```

from pyspark import SparkContext
from math import sqrt

sc = SparkContext("local[*]", "RideDispatch")

drivers = [
    ("D101", 13.0827, 80.2707, "Available"),
  
```

```
("D102", 13.0674, 80.2376, "Busy"),  
("D103", 13.0878, 80.2785, "Available"),  
("D104", 13.0750, 80.2600, "Available")  
]
```

```
driver_rdd = sc.parallelize(drivers)
```

```
rider_lat = 13.0800
```

```
rider_lon = 80.2750
```

```
available = driver_rdd.filter(  
    lambda d: d[3] == "Available"  
)
```

```
def calculate_distance(d):  
    return (  
        d[0],  
        sqrt(  
            (d[1] - rider_lat) ** 2 +  
            (d[2] - rider_lon) ** 2  
        )  
    )
```

```
distances = available.map(calculate_distance)
```

```
nearby = distances.filter(  
    lambda x: x[1] < 0.02
```



```
)

ranked = nearby.sortBy(
    lambda x: x[1]
)

print("Nearby Drivers:")
for driver in ranked.collect():
    print(driver)

best_driver = ranked.first()

print("Selected Driver:", best_driver)

sc.stop()
```

9. Advantages

The PySpark RDD-based approach provides:

1. **Parallel processing** – Multiple driver records can be processed simultaneously.
2. **Scalability** – The system can handle large numbers of drivers and ride requests.
3. **Fault tolerance** – RDDs can recover lost partitions through lineage.
4. **Fast filtering** – Unavailable and distant drivers can quickly be removed.
5. **Flexible matching** – Additional factors can be included in the matching score.
6. **Distributed computation** – Location calculations can be distributed across multiple worker nodes.

10. Limitations

The simplified implementation has some limitations:

- Latitude/longitude Euclidean distance is only an approximation.
- It does not calculate actual road distance.
- Traffic conditions are not considered.
- Driver ratings and vehicle types are not included.
- It processes static data rather than a complete real-time streaming system.
- Commercial ride-hailing systems use much more sophisticated optimization and machine-learning techniques.

11. Conclusion

The behavior of a ride-hailing dispatch system can be reverse engineered as a distributed data-processing pipeline. **PySpark RDDs** can represent driver and rider location data and perform filtering, distance calculation, ranking, and aggregation in parallel. The system first identifies available drivers, calculates their proximity to the rider, ranks suitable candidates, and selects the best driver for dispatch. Although this implementation is a simplified model of a commercial system, it demonstrates how distributed AI and big-data processing can be used to build a scalable ride-matching system.

```
PythonProject16 Version Control
Current File main.py
Run main.py

=====
DISPATCH RESULT
=====
Ride ID      : R001
Selected Driver : D101
Distance     : 0.005077
Driver Rating : 4.7
Dispatch Status : SUCCESS

=====
DISTRIBUTED PROCESSING PIPELINE
=====

DRIVER LOCATION DATA
|
v
CREATE RDD
|
v
FILTER AVAILABLE
|
v
CALCULATE DISTANCE
|
v
FIND NEARBY
|
v
RANK DRIVERS
|
v
SELECT BEST DRIVER
|
v
```

```
PythonProject16 Version Control
Current File main.py
Run main.py

v
FILTER AVAILABLE
|
v
CALCULATE DISTANCE
|
v
FIND NEARBY
|
v
RANK DRIVERS
|
v
SELECT BEST DRIVER
|
v
DISPATCH

=====
RDD OPERATIONS USED
=====

1. parallelize() -> Create distributed dataset
2. filter()      -> Select available drivers
3. map()         -> Calculate distance
4. filter()      -> Find nearby drivers
5. sortBy()      -> Rank drivers
6. first()       -> Select best driver

Project execution completed successfully.

Process finished with exit code 0
```

```
PythonProject16 Version Control
Current File
main.py
Run
C:\Users\rst290\PycharmProjects\PythonProject16\venv\Scripts\python.exe C:\Users\rst290\PycharmProjects\PythonProject16\main.py

=====
REVERSE ENGINEERING A RIDE-HAILING
DISPATCH SYSTEM
=====

RIDER REQUEST
-----
Ride ID : R001
Rider Location: 13.08 80.275

RDO CREATED
-----
Total driver records: 6

AVAILABLE DRIVERS
-----
D101 | Location: (13.0827, 80.2707) | Rating: 4.7
D103 | Location: (13.0878, 80.2785) | Rating: 4.9
D104 | Location: (13.075, 80.26) | Rating: 4.6
D105 | Location: (13.09, 80.285) | Rating: 4.8

NEARBY DRIVERS
-----
Driver: D101 | Distance: 0.005077 | Rating: 4.7
Driver: D103 | Distance: 0.008549 | Rating: 4.9
Driver: D104 | Distance: 0.015811 | Rating: 4.6
Driver: D105 | Distance: 0.014142 | Rating: 4.8

RANKED DRIVERS
-----
1. Driver D101 | Distance: 0.005077 | Rating: 4.7
2. Driver D103 | Distance: 0.008549 | Rating: 4.9
3. Driver D105 | Distance: 0.014142 | Rating: 4.8

PythonProject16 main.py
241:53 CRLF UTF-8 4 spaces Python 3.13 (PythonProject16)
19:41
06-09-2026
```